

COMP2200/COMP6200 Practical Exercise (Week 12): Recommender Systems with Graphs

School of Computing

LEU Survey

You have been invited to participate in a Learner Experience of Unit (LEU) survey for COMP2200/COMP6200 Data Science. You will have received an individual email invitation and link to the survey. You can also access the survey through the Student Feedback Surveys block on your iLearn homepage. Take some time to complete it now.

What this practical is about

This practical turns the recommender ideas from the lecture into a small working example. We will not use a large recommender library or matrix decomposition. The goal is to understand the evidence behind the recommendation.

By the end you should be able to:

- explain why popularity and content-based methods are useful baselines;
- turn ratings into a user-item graph;
- project a user-item graph into an item-item graph;
- recommend items by counting paths through shared users;
- compare path-counting recommendations with a simple random walk;
- test a recommender by hiding a known like and checking whether it comes back.

Preparation

Use the notebook `Week 12 Practical.ipynb`. It must be in the same folder as `simple_recommenders.py`.

1. Local with uv:

```
uv init
uv add pandas numpy matplotlib networkx notebook
uv run jupyter notebook
```

2. Google Colab: upload `simple_recommenders.py` into the Colab session before running the notebook cells. If a package is missing, install it with:

```
!pip install pandas numpy matplotlib networkx
```

Part A — A graph recommender by hand

Start with this tiny viewing table. A 1 means the user liked the item. A 0 means either they did not like it enough, or we have no useful signal.

User	Space	Dragon	Robot	Crime	Baking	Garden	Football	Bakery
Asha	1	1	1	0	0	0	0	0
Ben	1	1	0	0	0	0	0	0
Chris	0	0	0	1	1	0	0	1
Dana	0	0	0	1	1	1	0	0
Eli	0	0	1	0	0	0	1	0
Fiona	0	0	0	0	1	1	0	1

A1. Draw the user-item graph

Draw a bipartite graph with users on the left and items on the right. Add an edge wherever the table has a 1.

1. Which item has the highest degree?
2. Which users are in Ben's connected component?
3. Are there any disconnected parts of the graph?

A2. Count paths for Ben

Ben has liked **Space Quest** and **Dragon School**. We want to score things Ben has not already liked. For each candidate item, count paths of this form:

Ben -> liked item -> another user -> candidate item

1. How many paths lead from Ben to **Robot Detective**?
2. How many paths lead from Ben to **Baking Show**?
3. Which item should this graph recommender show first?
4. How confident should we be, given the size of the data?

A3. Popularity check

Path counting can quietly reward popular items. In a larger dataset, a blockbuster movie may appear near everyone simply because everyone has seen it.

1. How could you penalise very popular items?
2. When might a popularity penalty make recommendations worse?

Part B — Load the practical helper

Now move into the notebook. The helper functions keep the focus on the recommender behaviour rather than dataframe plumbing.

```
from simple_recommenders import (  
    build_user_item_graph,  
    graph_summary,  
    hide_known_like,  
    item_projection,  
    load_toy_items,
```

```

load_toy_likes,
load_toy_ratings,
projection_edges,
random_walk_recommendations,
ratings_matrix,
recommend_by_content,
recommend_by_popularity,
recommend_by_shared_neighbours,
recommend_by_similar_users,
related_items_from_projection,
score_hidden_like,
user_similarity,
)

```

Load the data:

```

items = load_toy_items()
ratings = load_toy_ratings()
likes = load_toy_likes(ratings, min_rating=4)

```

```
items
```

Inspect the ratings matrix:

```
ratings_matrix(ratings)
```

Question: What is the difference between a missing value, a low rating, and a like edge?

Part C — Baselines before graphs

Before using a graph, always check simple baselines. They are easy to explain and often hard to beat.

C1. Popularity baseline

```
recommend_by_popularity(ratings, items, user="Ben", top_n=5)
```

1. What evidence is this method using?
2. Why is it a reasonable first baseline?
3. Why might it be boring for Ben?

C2. Content-based baseline

Ben likes Space Quest and Dragon School. Recommend items with similar tags.

```

recommend_by_content(
    items,
    liked_items=["Space Quest", "Dragon School"],
    top_n=5,
)

```

1. Which tags seem to drive the recommendation?
2. Does this method need other users?
3. What can it recommend for a brand new item?

C3. Similar users

```
user_similarity(ratings, user="Ben")

recommend_by_similar_users(ratings, items, user="Ben", top_n=5)
```

1. Which users are most similar to Ben?
2. Is this recommendation more personal than popularity?
3. What happens if a user has no ratings yet?

Part D — Build the user-item graph

Ratings of 4 or 5 have been turned into like edges. This is a modelling choice, not a fact of nature.

likes

```
graph = build_user_item_graph(likes, items)
graph_summary(graph)
```

D1. Inspect the graph

```
sorted(graph.nodes(data=True))[:5]

sorted(graph.edges())[:10]
```

1. How many connected components are there?
2. Why might disconnected components be a problem for recommendations?
3. What other evidence could connect the components?

D2. Optional visualisation

This is small enough to draw. The layout will move around a little unless you keep the random seed fixed.

```
import matplotlib.pyplot as plt
import networkx as nx

pos = nx.spring_layout(graph, seed=42)
labels = {
    node: data.get("label", node)
    for node, data in graph.nodes(data=True)
}

plt.figure(figsize=(8, 5))
nx.draw_networkx(graph, pos=pos, labels=labels, node_size=900, font_size=8)
plt.axis("off")
plt.show()
```

Part E — Project to an item-item graph

The user-item graph answers questions about people and items together. For recommendations, it is often useful to build an item-item graph: two items are connected when at least one user liked both.

```
projection = item_projection(likes, items)
projection_edges(likes, items)
```

E1. Related items

```
related_items_from_projection(likes, "Space Quest", items, top_n=5)
```

1. Which item is most related to Space Quest?
2. Is the edge weight strong evidence or weak evidence here?
3. How would this change with thousands of users?

E2. Recommend by shared neighbours

```
recommend_by_shared_neighbours(likes, items, user="Ben", top_n=5)
```

This is the Python version of the hand calculation from Part A. The score is a path count through items and users.

1. Does it match your hand calculation?
2. How is this different from the content-based recommendation?
3. Which method would be easier to explain to a user?

E3. Try a popularity penalty

```
recommend_by_shared_neighbours(  
    likes,  
    items,  
    user="Ben",  
    top_n=5,  
    penalise_popularity=True,  
)
```

Question: Did the ranking change? In this tiny dataset, is the penalty helping or just adding noise?

Part F — Random-walk recommendations

A random walk samples paths instead of counting every path exactly. The lecture version used:

```
user -> liked item -> another user -> another item
```

Run the small random-walk recommender:

```
random_walk_recommendations(  
    likes,  
    items,  
    user="Ben",  
    walks=1000,  
    seed=42,  
    top_n=5,  
)
```

1. Does the random-walk result agree with the shared-neighbour result?
2. Change `walks` to 20, then 2000. What changes?
3. Change the `seed`. What changes?
4. Why might a random walk be useful on a very large graph?

Part G — Hide a known like and test it

Evaluation is not just “did the code run?” We need to hide something known, make recommendations without it, and check whether the recommender finds it again.

```
train_likes, hidden = hide_known_like(  
    likes,  
    user="Ben",  
    item_id="dragon_school",  
)
```

hidden

Build recommendations using only the training likes:

```
test_recs = recommend_by_shared_neighbours(  
    train_likes,  
    items,  
    user="Ben",  
    top_n=5,  
)
```

test_recs

Score whether the hidden item appears:

```
score_hidden_like(test_recs, hidden, k=5)
```

1. Did the hidden item appear in the top 5?
2. Is one hidden item enough to prove the recommender is good?
3. What would a fairer test look like?

Part H — Design decisions

Answer these in short notes.

1. **Cold start:** What should the system show a brand new user?
2. **New item:** Which method can recommend a new item before anyone likes it?
3. **Popularity:** Should the system downweight blockbusters? Why?
4. **Diversity:** Should the top five recommendations all be very similar?
5. **Transparency:** Which recommendation from today would be easiest to explain?
6. **Objective:** What changes if the platform optimises for watch time rather than satisfaction?

Optional extensions

1. Add a new item to `load_toy_items()` with tags but no ratings. Which recommender can use it immediately?
2. Add yourself as a new user with three ratings. Compare popularity, content-based, shared-neighbour, and random-walk recommendations.
3. Change the like threshold from 4 to 3 in `load_toy_likes`. How does the graph change?
4. Hide a different known like. Does the recommender still recover it?
5. Write a product rule that combines graph score, content score, and one diversity rule.

Takeaway

A recommender is not just a clever model. It is a product decision made from evidence. Today the evidence was ratings, item tags, user-item links, shared neighbours, and sampled graph walks. In a real system the data is bigger and messier, but the questions are the same: what evidence are we using, what are we optimising, and what mistakes will that create?